# START TREK Report - Reinforcement Learning

*Ariel Amriou, Pierrick Simon, Sacha Lemée*

## 1. Introduction

### 1.1 Contexte

Le projet START TREK a pour objectif de développer un agent autonome capable d'apprendre à faire atterrir un module lunaire, dans l'environnement LunarLander-v3 de Gymnasium.

L'apprentissage ne repose pas sur des règles codées manuellement. L'agent découvre progressivement les bonnes actions à effectuer grâce à un apprentissage par renforcement.

À chaque épisode :

- l'environnement fournit un état
- l'agent choisit une action
- l'environnement retourne une récompense
- l'agent observe le nouvel état
- et l'apprentissage continue jusqu'à la fin de l'épisode

Le projet permet de mettre en pratique plusieurs concepts fondamentaux du Reinforcement Learning :

- exploration vs exploitation
- fonction de valeur
- politique de décision
- apprentissage temporel
- approximation par réseau de neurones ou par stockage environnement / action

L'environnement utilisé est le benchmark officiel Gymnasium LunarLander-v3.

## 1.2 Problématique

L'objectif principal du sujet est le suivant :

> Comment apprendre automatiquement à un module lunaire à atterrir au travers d'un apprentissage par renforcement ?

Le problème présente plusieurs difficultés :

- Espace d'observation continu
- Dynamique physique complexe
- Equilibre d'exploration/exploitation
- Convergence parfois instable

Le critère de réussite de nos modèle sont alors d'obtenir une moyenne supérieure ou égale à 200 de récompense sur 100 épisodes consécutifs



# 2. Implémentation du Reinforcement Learning

## 2.1 Contexte et environnement

L'environnement LunarLander-v3 fournit un état composé de 8 valeurs :

- position x / y (de -2,5 à 2,5)
- vitesse x / y (de -10 à 10)
- angle du module (de -7.5 à 7.5)
- vitesse angulaire (de -10 à 10)
- contact jambe gauche (0 ou 1)
- contact jambe droite (0 ou 1)

L'agent possède 4 actions possibles :

| Action | Description |
|---|---|
| 0 | Ne rien faire |
| 1 | Activer moteur central |
| 2 | Activer moteur gauche |
| 3 | Activer moteur droit |

La boucle du Reinforcement Learning suit toujours le même schéma :

| état -> action -> récompense -> prochain état |
|---|

### 2.1.1 Concepts de Reinforcement Learning utilisés

**Politique**

La politique représente la stratégie de décision de l'agent, elle permet de répondre à la question suivante: quelle action vais-je faire dans cet état.

**Exploration vs Exploitation**

Afin que le model apprenne constamment, il doit choisir entre ses deux choix:

- explorer de nouvelles actions aléatoires
- exploiter les meilleures actions déjà apprises

Pour garder un équilibre entre exploration et exploitation, nous utilisons ici la stratégie ε-greedy, qui consiste en ayant une probabilité d'exploration qui évolue à la baisse afin qu'il utilise de plus en plus les valeurs apprises.

Configuration utilisée :

| |
|---|
| espilon_start*: 1.0<br>espilon_end**: 0.05<br>espilon_decay***: 0.995 |

* chance d'exploration au début de l'entraînement
** chance d'exploration à la fin de l'entraînement
*** facteur de dégradation de la chance d'exploration

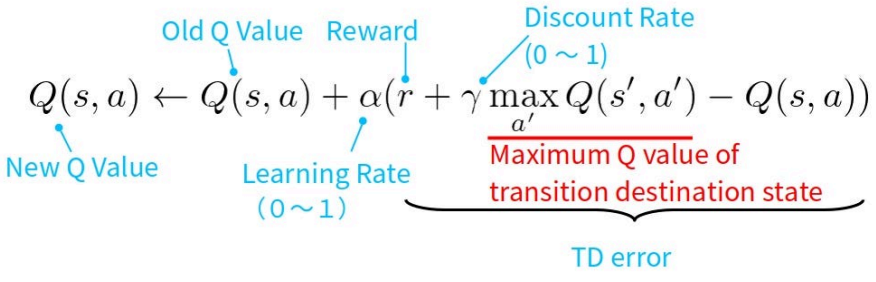
## 2.2 Agent Q-Learning

### 2.2.1 Implémentation

Le premier agent repose sur un Q-Learning classique, model utilisant un table Q afin de stocker les possibilités d'action dans des états différents et leurs poids associés.

L'environnement étant partiellement continu, une étape de discrétisation est obligatoire pour avoir une table Q d'une taille définie (exemple: 2.1 -> 2).

La mise à jour de la Q-table suit l'équation de Bellman:




$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$

### 2.2.2 Améliorations

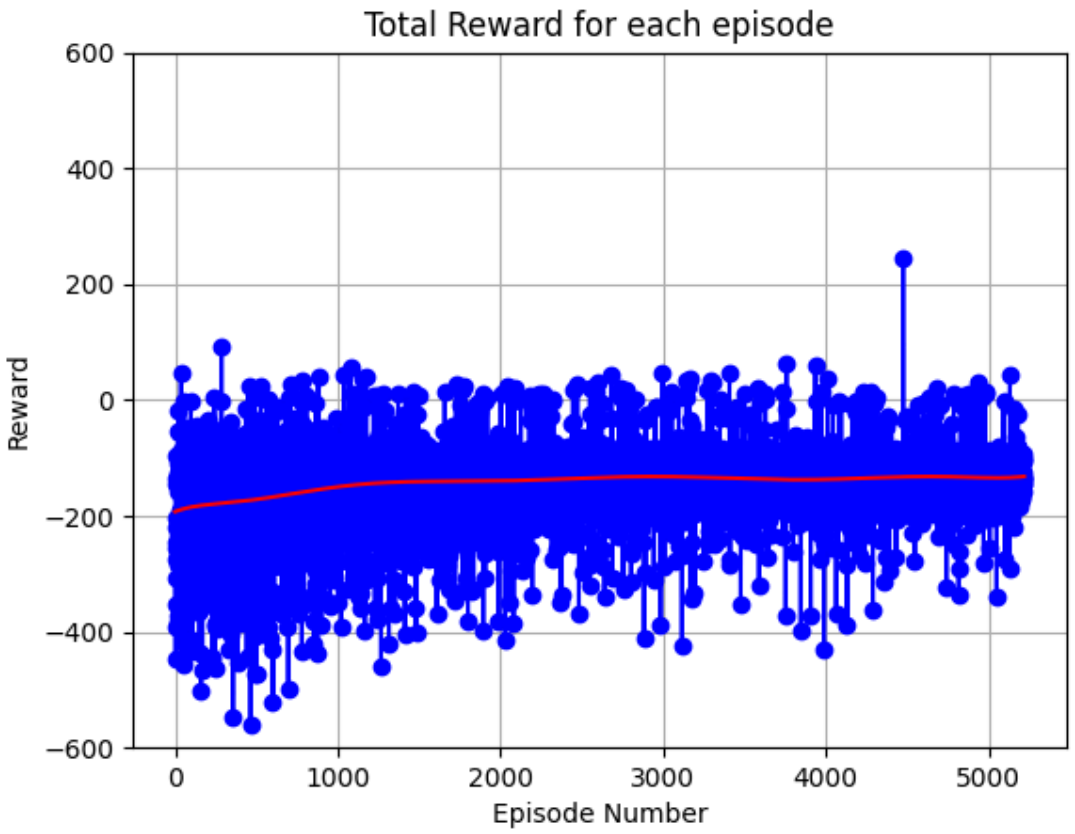
Plusieurs améliorations ont été ajoutées :

- ε-greedy décroissant
- sauvegarde automatique de la Q-table
- discrétisation configurable
- zoom de l'espace d'état

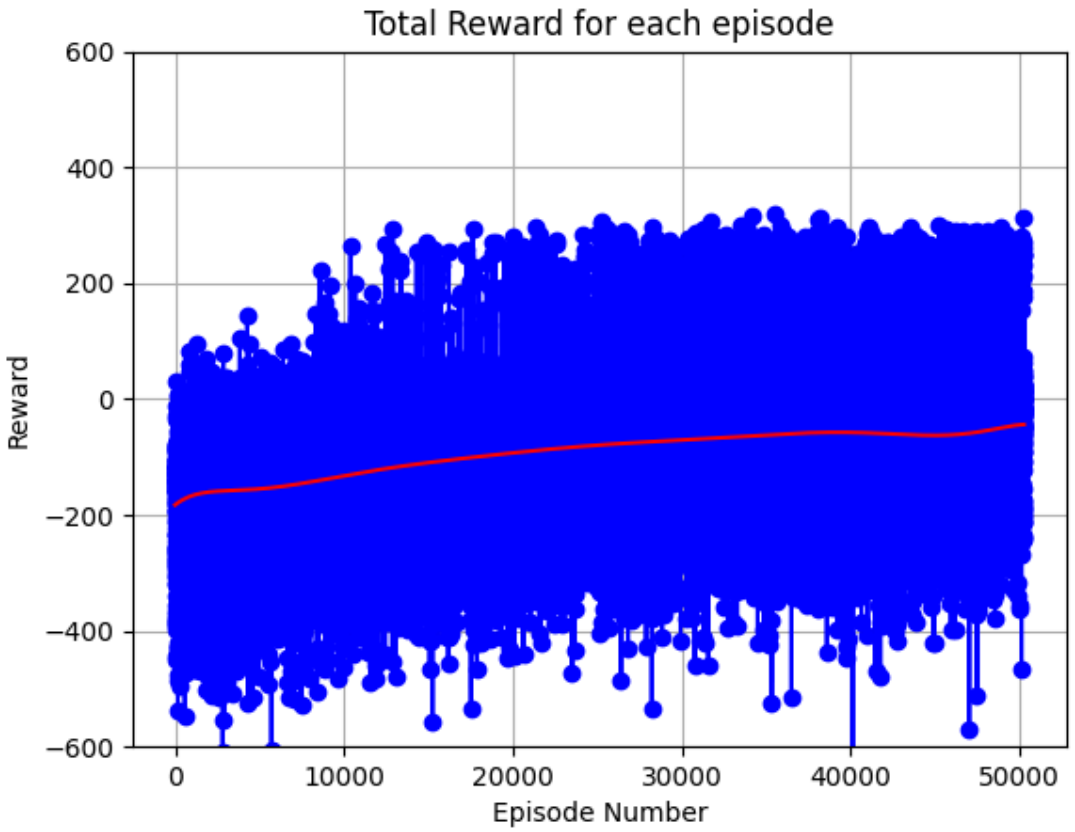
L'environnement étant partiellement continu, une étape de discrétisation personnalisé est obligatoire. Il est alors possible de changer de gamme pour améliorer le contexte de notre modèle.

### 2.2.3 Résultats

Résultats du Q-learning sans méthodes d'optimisations:



Résultats du Q-learning avec méthodes d'optimisations (cités ci-dessus):




Selon les graphiques ci-dessus, il est possible de voir que le modèle sans optimisation est très ressemblant au modèle de comparaison aléatoire, alors que celui avec des méthodes d'optimisation apprend très lentement mais augmente de moyenne.

## 2.3 Agent Deep Q-Learning

### 2.3.1 Implémentation

Ce second modèle, au lieu d'utiliser une table Q, utilise un réseau de neurones ou les Q v alues sont approximées et ou les valeurs restent continues.

### 2.3.2 Méthode de Dueling

Afin de la variance et améliore la stabilité de l'apprentissage, nous avons utilisés une méthode de Dueling DQN, qui sépare :

- la valeur globale d'un état
- l'avantage spécifique de chaque action

Le calcul final est donc :

$$Q(s,a)=V(s)+A(s,a)-\text{mean}(A(s,a))$$

### 2.3.3 Replay memory

Le DQN utilise également une Replay Memory.

Chaque transition est stockée de la manière suivante :

```
(state, action, reward, next_state)
```

Les groupes sont ensuite échantillonnés aléatoirement.

Cela permet de :

- stabiliser l'apprentissage
- améliorer la convergence

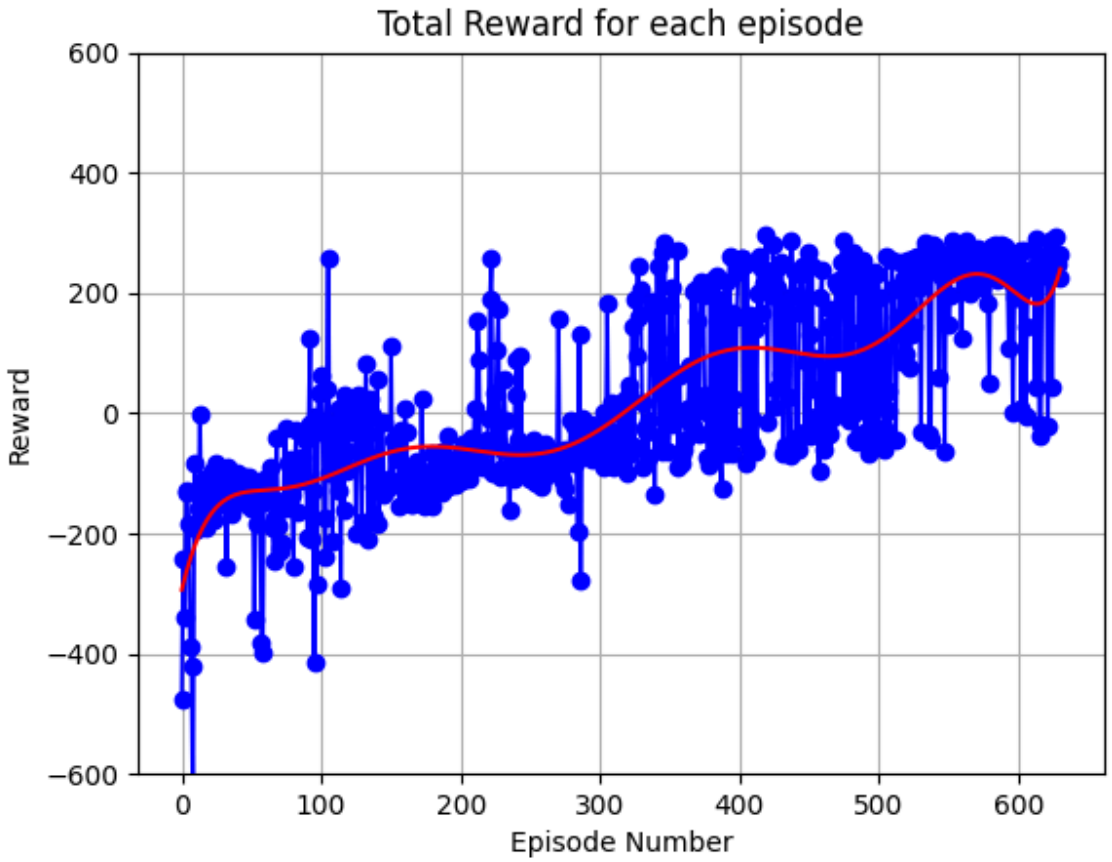
Configuration utilisée :

```
memory_capacity*: 50000
batchsize**: 64
target_update***: 1000
```

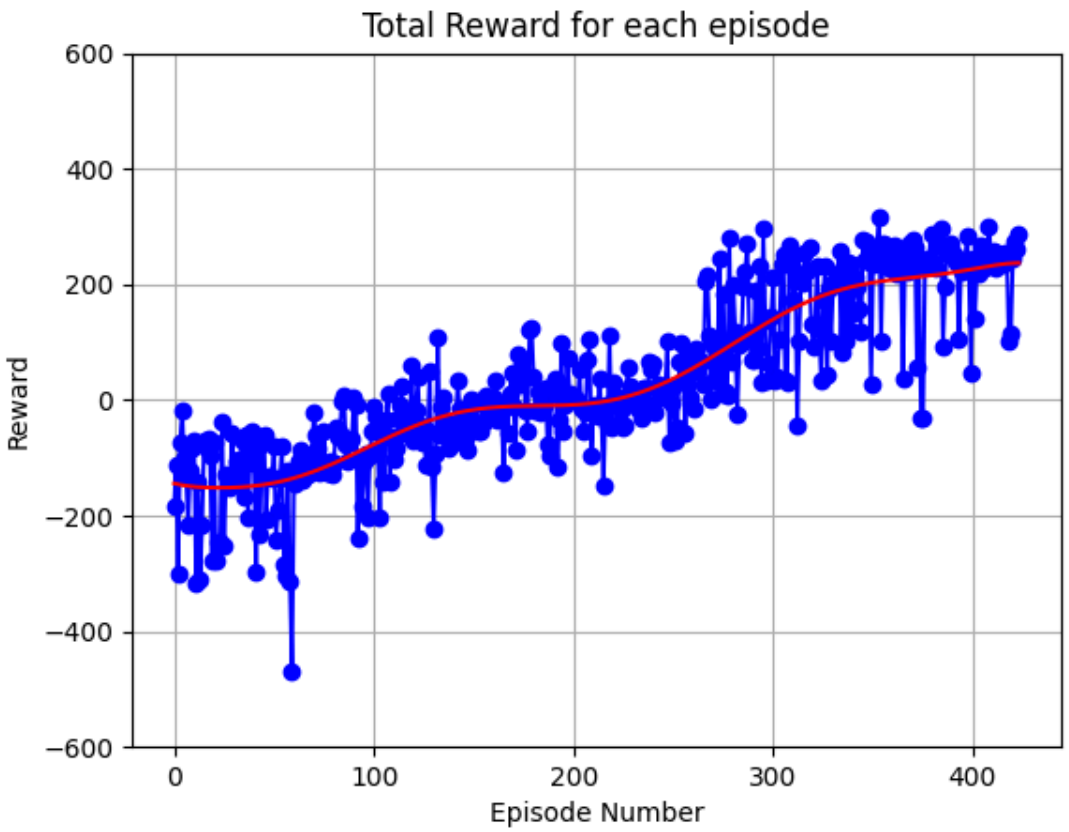
* capacité de la mémoire
** nombre de donnés utilisés pour l'apprentissage
*** rapidité d'actualisation du réseau de neurones

### 2.3.4 Résultats

Modèle de Deep-Q Learning sans optimisation (45 min):



Modèle de Deep-Q Learning avec optimisation (10 min):




## 3. Tests réalisés et interprétation des résultats

### 3.1 Etude d'ablation

#### Impact du Learning Rate

| Learning rate | Résultat |
|---|---|
| 0.001 | Instable |
| 0.0001 | Stable |
| 0.00001 | Trop lent |

Le meilleur compromis observé est donc 0.0001.

**Impact de la Replay Memory**

| Taille mémoire | Effet |
|---|---|
| 1000 | Corrélation forte |
| 50 000 | Stable |
| 100 000 | Peu d'amélioration |

La capacité de 50 000 transitions offre le meilleur équilibre mémoire/stabilité.

**Impact du epsilon decay**

Un epsilon trop élevé :

- ralentit fortement la convergence

Un epsilon trop faible :

- bloque l'agent dans des minima locaux

Le decay 0.995 donne une transition progressive correcte.

### 3.2 Analyse qualitative

**Q-Learning**

Le comportement observé ressemble à :

- Des mouvements brusques causés pas la discrétisation
- Des scores non stables
- Des scores parfois extrêmement bas
- Un entraînement très lent avec beaucoup d'épisodes (30 minutes et 10000 épisodes)

**DQN**

Le réseau apprend progressivement :

- Des corrections et un atterrissage plus douces
- Une meilleure anticipation
- Un entraînement plus rapide avec très peu d'épisodes (10 minutes et 500 épisodes)

## 4. Conclusion et perspectives

Ce projet nous a permis d'implémenter deux approches majeures du reinforcement learning qui sont chacun capables d'apprendre à atterrir au fils du temps:

- Q-Learning
- Deep Q-Network (DQN)

Le DQN offre néanmoins de meilleures performances :

- meilleure stabilité
- meilleure généralisation
- adaptation naturelle à un espace continu

### 3.1 Perspectives d'amélioration

Avec plus de temps à notre disponibilité il aurait été possible d'implémenter les fonctionnalités suivantes:

- Apprentissage plus long
- Changement des hyperparamètres
- Activation du vent dans l'environnement
- Comparaisons avec d'autres modèles (Sarsa, PPO, ou encore TRPO)

# 5. Annexes

## 5.1 Structure du projet

```
.
├── configs
│   ├── dqn.yaml
│   ├── qlearn.yaml
│   └── random.yaml
├── data_dqn
│   ├── csv
│   ├── images
│   ├── model
│   └── videos
├── data_qlearn
│   ├── csv
│   ├── images
│   ├── model.npy
│   └── videos
├── data_test_model
│   ├── csv
│   ├── images
│   └── model.npy
├── doc
│   ├── example.yaml
│   ├── G-AIA-401_start_trek.pdf
│   └── lunar_lander.gif
├── LICENSE
├── README.md
├── reproduce.sh
├── requirements.txt
├── review.py
├── src
│   ├── qlearn.py
│   ├── random.py
│   ├── termination.py
│   ├── train.py
│   └── video.py
└── startTrek.py
```

Lien vers la documentation Gymnasium : Gymnasium Documentation

Lien vers le repository : Start Trek Repository